

Improving General Knowledge in Agile Software Organizations

Experiences with job rotation in customer support

Tor Erlend Fægri

Department of Software Engineering, Safety and Security

SINTEF ICT

Trondheim, Norway

Tor.E.Fægri@sintef.no

Abstract—For many organizations the transition to agile methods is problematic due to history of bureaucratization and subsequent extensive specialization of knowledge among people. Specialist knowledge inhibits self-organization and role interchangeability which are key elements of agile development. Knowing that bureaucracies are hard to counteract once established, how can development of general knowledge in software organizations be improved? Job rotation is a well-known practice often used to improve general knowledge. The reported action research evaluated job rotation among developers in customer support. The findings suggest that general knowledge is considered interesting and valuable among the participants. However, the findings also show that general knowledge acquisition can be found irrelevant and therefore counter-efficient for day-to-day work among participants if the perceived applicability to own projects is too low. Therefore, using job rotation to improve general knowledge requires careful considerations. Implications for research and practice are discussed.

Keywords—Agile methods, adoption of agile methods, agile organizations, job rotation, organizational learning

I. INTRODUCTION

Agile Software Development (ASD) is based on self-organizing teams for which general knowledge is important. Job rotation is one approach to build general knowledge. This action research study reports upon experiences from a trial of job rotation among developers in CS (Customer Support) at a Norwegian software organization henceforth called MapIT (a pseudonym).

The MapIT software organization employs roughly 30 people (16 developers) and is a provider of electronic document management systems, geographical information systems and a set of related services such as consulting and service hosting. Development activities are predominantly allocated to projects with budget and time constraints. Developers can be grouped according to technology; those working primarily with Java (5 developers) and those working with .NET (11 developers). Document management applications are mostly based on .NET while graphical applications dealing with maps etc. are mainly written in Java. Over the last two years MapIT has been in the transition from plan-driven to agile software development, using Scrum. Both developer groups actively pursue agile

development and both groups are located in the same office space.

To support readability of quotes we use pseudonym names preceded with codes in parentheses for showing affiliation to different groups. Code “J” denote Java group, code “N” denote .NET group, code “S” denote permanent CS – i.e. 1st line CS, and code “M” denote management group. For example, David (J) denotes a statement from David belonging to the Java group. Tracy (J & N) denotes statements from Tracy which has some experience from the .NET group but is now part of the Java group.

Prior to the trial there was only one staff, Carl (S), working in CS. He acted mostly as a dispatcher of customer enquiries (1st line CS). The organization’s product portfolio was large and diverse with clear potentials for improved technical alignment. Some products were developed by single developers. For the larger products the responsible developers had frequently non-overlapping ownership of the products’ modules. Thus, when a CS enquiry arrived it was normally simply dispatched to the developer that had worked on the particular product or module associated with the enquiry. Naturally, this caused a lot of disruptions to the daily workflow of developers. Dealing with CS enquiries was called ‘the quagmire’ to illustrate their dismay of being personally responsible for this work on top of other duties. The symbolic name ‘quagmire’ also reflects the difficulty of developers to withdraw from personal customer responsibility.

The paper is organized as follows: Section II elaborates on the action research approach, our collaboration with MapIT and our motivation for using action research. Section III presents a summary of existing research regarding specialist knowledge and general knowledge in organizations with a particular attention to the importance of general knowledge in agile software development. Section IV gives a summary of the empirical findings. Subsequently, Section V discusses the contribution of the findings to existing research. Section VI contains our conclusions.

II. RESEARCH APPROACH

The research is based in action research [3, 8, 13] and organizational learning [2]. Before this particular

investigation we had a history of long-lasting collaboration with MapIT on the topics of software process improvement and adoption of agile methods. Thus, MapIT was in a good position to cooperate with the researchers in trying to deal with any problems. The epistemological basis of action research is that researchers and practitioners collaborate in order to improve current practice while also contributing to existing theory. Due to our history of collaboration with MapIT the researchers had collected extensive knowledge on many challenges at MapIT and could more easily obtain a suitable position to engage in organizational change processes. However, our engagement during the action taking phase was facilitative [3] rather than collaborative [13]. We did engage closely during diagnosing, planning, evaluation and specifying learning. Thus, as action research it is best classified as participant action research [29].

During the spring 2008 the researchers had four meetings with MapIT to diagnose the nature of the problem and to plan possible actions to correct it. Trying to reduce the 'quagmire' problem quickly became the focus of the research project. From these four meetings with MapIT we learned a great deal about how the problem was perceived. The MapIT general manager, Nick (M), referred to the quagmire as "the cause of disruptions in the machine" implying that MapIT failed to live up to bureaucratic ideals [33]. Peter (M), the current consultant manager, warned that MapIT was "unable to learn from mistakes" which suggested that organizational learning potentials were not exploited [2]. Mark (M), the current development manager, commented: "We are too comfortable living with our mistakes." The latter statement suggested that there were organizational conflicts hiding beneath the surface. It became clear that MapIT needed a different way to deal with CS enquiries. Furthermore, there were concerns of a very wide and poorly technically aligned product portfolio. We got no precise answers to questions on the number of products, but guesses fell within the 15-35 range. A developer referred to an example where seemingly identical functionality for document scanning had been developed in two separate products. These issues suggested that MapIT did not exploit its learning potentials [2, 12].

After some time for reflection the idea of trying job rotation to CS came up. The researchers argued that if developers rotated to an office collocated with Carl (S) this could be a way of creating a rich learning environment for improved general knowledge of MapIT's range of products. It would also allow Carl (S) to learn more about developers' way of solving problems. It would simultaneously give shielding for the other developers for CS enquiries. There were critical comments from some of MapIT managers to this approach however. Primarily, the concerns were related to the ability of a rotation system to ensure high quality CS to customers in case rotation introduced additional handovers between people. Therefore the trial period would be relatively short with formal evaluation in September 2008.

From the action-science perspective the investigation can be viewed as an experiment in enabling double-loop learning; how can fundamental practices be altered to improve symptomatic challenges? In this specific case; developers were too busy working on their separate products

to learn about other products. Essentially, they focused on building specialist knowledge for a small set of products. Thus, it was often difficult to share the load of CS work among developers. Furthermore, a range of learning opportunities that could help improve the future technical alignment within the product portfolio was missed.

The primary objective of the investigation was to understand more about the premises for organizational learning in software organizations through job rotation. These are inherently complex phenomena on their own [2, 23, 24] and should subsequently be assumed to involve further complexities when investigated together. Furthermore, research on ASD is still in a nascent state [9] which motivate adaptive research methods suited for exploring and uncovering new phenomena, and new relationships between the phenomena [10]. Action research was therefore found to be a suitable research approach as it is inherently targeted at developing understanding through collaboration and change.

The specific research questions underpinning this investigation are: A) How can job rotation contribute to the building of general knowledge in an organization with a prevalence of specialist knowledge? B) What impact can the introduction of job rotation have on enabling double-loop learning in a software organization? C) Can the introduction of job rotation illuminate other challenges related to organizational learning in software organizations?

The job rotation schedule and a dedicated CS room were ready early May 2008 which also marked the start of the actual trial. This room had four desks facing each other in a diamond shape and large windows to approximate the sensation of working in an open environment, but had a glass door that could be closed to reduce noise from the surroundings. Most of the time, we would find the door open though. The room was located at the opposite end of the office space, together with another set of offices, roughly 20 meters from the main development area. The rotation schedule defined when developers should sit in the CS room and participate in the CS work as 2nd line CS together with Carl (S). For the following 18 weeks MapIT maintained the job rotation practice. Initially the rotation schedule had intervals of two weeks to coincide with the Scrum iterations. This was later reduced to one week to accommodate dismay of excessive project delays. Also, initially the rotation schedule included one developer from the Java group and one developer from the .NET group to participate in the CS function simultaneously. This was later changed to only one developer, picked from either the Java or .NET group. The reason, we learned during our evaluation, was to reduce cost.

In the 5th week of the trial we conducted a preliminary review meeting where all participants and management was invited. There were critical opinions towards the rotation; many developers felt they were wasting time trying to deal with problems related to projects they hadn't intimate knowledge about. However, they also noted that the Wiki had appeared as a useful knowledge sharing tool and that the time in CS gave opportunities for contributing useful information to the Wiki. Besides, the rotation participants

liked the idea of learning about the range of products in the portfolio. Management decided to continue the trial, and the meeting concluded with an agreed schedule for concluding interviews and presentation of evaluation results for MapIT. The trial continued for 13 more weeks (i.e. 18 weeks in total). At the end 10 people, including 1st line CS, had been participating in the trial. All 10 were interviewed using semi-structured interviews to gather opinions on job rotation, CS work and the subjects' own reflections on development of general knowledge through rotation. In addition we conducted 6 other semi-structured interviews of people more peripherally involved (one non-participating developer, and five people with management roles.) These 6 interviews were based on a reduced version of the participants' interview guide.

Action research embraces both qualitative and quantitative analysis – or a mix. The 16 interviews mentioned, totaling more than 7 hours, and 77 A4 pages of transcribed text, constitute the main source of data. The second source of data is logs from CS work. The logs were filled in by all involved in CS work and captured information on individual CS enquiries such as involved persons, enquiries resolved by different roles (1st, 2nd and 3rd line). The third data source is notes from the review meeting. Analysis of the interviews, meeting notes, and CS logs were supported by tools for qualitative data analysis and spreadsheets. Interviews and meeting notes were immediately transcribed and carefully analyzed to support reliability. Only log data captured during the trial were available. Furthermore, as the duration of the trial was short, the complexity of the measured phenomenon is high and involves learning which is inherently dependent on experience and accumulation of knowledge the log data are used to support only the argument regarding shielding.

A few weeks after our concluding interviews early September 2008, job rotation was abandoned and one of the trial participants was appointed permanent 2nd line CS.

III. THEORETICAL BACKGROUND

Efficiency through bureaucratization has been a fundamental basis of western management and organization discourse since the successes of Taylor in the 1910s and the Ford mass production practices in the 1920s [6]. It has also greatly influenced software engineering practice [21]. An important aspect of bureaucratization is to reduce variability in performance by limiting human judgment and rather transfer power to governing systems seeking to plan the work through procedures and normative rules [33]. It can be difficult to argue against this approach to improve efficiency. The bureaucratic organization of work represents optimization by encoding knowledge and prior experience into a stable, fair and predictable machine-like structure. Besides, the efficiency perspective of bureaucracy is based on rational conduct which is both logical and sensible as well as reasonable and adherent to scientific ideals [6]. Furthermore, once established bureaucracy is inherently difficult to reduce [33] as it requires changes in mindset and culture [21].

The bureaucratic organization assigns people to specific roles or functional areas to attain high specialized role performance and pursue particular careers [15]. While benefitting people's individuality it gives rise to specialist knowledge because people focus on a certain range of tasks and these tasks constitute their fundamental learning environment [6]. Specialist knowledge is knowledge that is concentrated among a few individuals [25]. Increasing complexity of software systems may easily lead people to focus their attention to particular modules, technologies or aspects of the software system being developed, primarily due to cognitive limitations of the brain [12, 24]. Contrary to specialist knowledge, general knowledge is knowledge that covers and somehow integrates multiple knowledge domains [25]. General knowledge is also called common knowledge [5] or breadth knowledge [12, 34]; implying that the knowledge is wide and at an overall level. The aspect of general knowledge that it somehow connects, or integrates multiple fields of understanding is explained by Kolb [15]. In his view, integrative knowledge is the third stage of a knowledge pyramid, above acquisition and specialization (p.140). He suggests that the integrative type of knowledge is the pinnacle of human learning and helps us to deal with "the interface between social knowledge and the ever-novel predicaments and dilemmas we find ourselves in." (p.225).

A fundamental premise for achieving efficiency through bureaucracy is a relatively slow rate of change in the organization's environment. This is often not the case for software organizations [20] for which the benefits of extensive planning and bureaucratization are reduced [17]. Since the 1970s evolutionary development approaches have been promoted to constitute an alternative to former plan-driven, bureaucratic software engineering practices [16]. During the recent years, ASD (Agile Software Development) has become a widely accepted method that builds on ideas from evolutionary development. The efficiency perspective in ASD is different from that of plan-driven development. For example, ASD promotes flexibility instead of rigidity, for example through self-organizing teams and role interchangeability [21]. Self-organizing teams take more responsibility for their own work and reduce the need for bureaucratic, hierarchical management [4]. Also, ASD emphasizes work-coordination through tacit knowledge sharing rather than written documents which also restrict management insight to the team [21]. A pre-requisite for efficiency in teams is general knowledge, such as common understanding of goals and problem-solving strategies [27, 34]. General knowledge, in the form of common knowledge [6] or shared domain knowledge [18] functions as an enabler for effective communication. General knowledge is held to be the pre-requisite for backup behavior in teams [27]. Specialist knowledge, on its own, does not constitute competitive advantage before it is somehow integrated or shared [12] and thus made available to benefit the activities of the organization.

Numerous challenges to the successful adoption of agile methods have been identified in literature [4, 21]. One is existing prevalence of specialist knowledge among the team members that can inhibit self-organization [19]. For many

organizations, it can be difficult to alter fundamental forces that stimulate development of specialist knowledge. Considering cognitive limitations promoting specialization [12], people's desire for achieving proficiency and high performance in their job situation [15], bureaucracy as a traditional form of efficiency [21] and that through increasing experience the specialist easily becomes the preferred person to deal with a certain type of tasks, a natural tendency would appear to favor development of specialist knowledge.

ASD include recommended practices that are intended to increase the level of general knowledge at the team level. Examples are group-based estimation, collective code ownership and pair programming. Job rotation is another well-known practice to build general knowledge beyond the team level. Job rotation is normally used to denote an organization of work by which people specialize in different roles according to some schedule [7]. Although job rotation represents formalism it is one which seeks to break up fallacies of traditional bureaucracy. Ortega suggests that job rotation and specialization denote the two extremes of a job policy dichotomy [23]. Besides, organizational routines can be effective mechanisms for knowledge sharing [12].

Literature discusses several benefits of job rotation. It is a characteristic of learning organizations that may contribute to higher innovative performance [17], and be beneficial to tacit knowledge sharing [30] and knowledge creation [28] through development of knowledge redundancy and 'requisite variety' [22] which is critical for knowledge intensive work such as software development. Job rotation can improve team efficiency [14], problem-solving skills [1] and can enable people to understand larger aspects of an organization's processes [7]. Subsequently it can benefit exploration of new knowledge [31]. Job rotation, through its ability to create individual cross-functional experience, can also be superior to e.g. cross-functional teams in order to build general knowledge because people tend to prefer to talk about already shared knowledge [11]. But most of the identified research has focused on productivity effects rather than adoption challenges and peoples' experience. Un [31] proposes that job rotation increased psychological safety among participants. Empirical evidence is scant however [23]. Within the field of software engineering, job rotation has simply been encouraged [26]. Within agile software development there is barely anything [9]; Wang [32] suggests that job rotation to CS can contribute to developer motivation while providing a valuable source of customer requirements (pp.155, 165).

Improved understanding of job rotation participants' experiences can be valuable in determining useful strategies for building general knowledge and sustaining organizational learning in agile software organizations. Subsequently, such understanding can improve organizations' adoption of ASD.

IV. FINDINGS

At the review meeting conducted 5 weeks into the trial the developers considered the job rotation as mainly problematic. Most of the developers participating in the

meeting did not see value in rotation to CS. One reason was that the amount of 'personal ownership' of code among the different products. One developer noted that the .NET group and the Java group had their 'individual ways' to solve problems which created trouble when trying to support products from the other technological camp. Also, a couple of the developers allocated for CS shifts had been able to 'slip away' without doing the shift and thereby causing concerns for lack of fair sharing. Another reason was poor tools that did not allow for efficient tracking of bugs among multiple customers. However, there were also a few developers that noted that the trial should be allowed to run for a while before formal evaluation. One noted that such learning initiatives take a long time before giving visible effects.

The researchers didn't intervene for the following 13 weeks. At this point we conducted the formal interviews. Now, Karen (M) had just recently replaced Anthony (M) as development manager. Prior to this, Anthony (M) had acted as temporary development manager while MapIT was looking for a replacement for Mark (M). Only a couple of weeks earlier Ken (M) had replaced Peter (M) as consultant manager.

A. Attitude to job rotation

The interviews revealed that only a minority of the participating developers in the job rotation trial (3 out of 9) was willing to continue with the current practice of job rotation into CS. 4 out of 9 were strongly against. However, the remaining 2 participant would accept job rotation if 2nd line CS was permanently staffed, and thus result in rotation to a 3rd line CS function within the room. In fact, all of the 9 participating developers would prefer a permanent 2nd line CS staff. There was a strong correlation between developer group and attitude towards rotation; the 4 against rotation belonged to the Java group. Carl (S), permanent 1st line CS staff, was also in favor of a permanent 2nd line CS and expressed uncertainty about any kind of job rotation. These results caused MapIT management to appoint one of the participating developers to become a permanent 2nd line CS staff, sitting together with Carl (S). Job rotation was abandoned.

The interviews with management suggested a more neutral attitude to rotation as such. But it was noted that the developers had different motivations for support work. Anthony (M), filling the role of development manager during the trial, explained: "It depends a lot on the person. Some people are very conscientious and don't want to abandon their commitment to customers in the middle of a Scrum project. While others are more willing to do just that." Then there were concerns that fewer people on rotation might be better for maintaining focus and efficiency. Ken (M), the current consultant manager, was strongly against rotation though. He had prior experience from managing a CS unit in another organization and believed that job rotation could never provide sufficiently professional handling of CS work.

B. On support work

Several important criticisms were revealed in the interviews. Some suggested that the job rotation practice was a cheap management trick introduced to avoid the cost of a permanent 2nd line CS staff. Besides, we learned that CS work had a lower status than development work – in particular among the less tenured developers which complained that CS work “was not part of the contract” and involves “the dealing with problems you didn’t create yourself.” Helen (N), a developer with more than 10 years of tenure painted a more nuanced picture: “Some think CS work is degrading, more akin to ‘fixing things’ and getting your hands dirty while development is more similar to the creative role of the architect. But in reality we as developers also get our hands dirty, for example when there are a lot of users involved that interfere with the way you originally conceived the solution.” We learned that mostly .NET applications were in use by end-users. So far, Java projects had developed technical infrastructure for use by other applications although end-user Web-applications were underway. This is likely an important factor for explaining the low motivation for job rotation to CS among Java developers. CS work was also associated with stress. Said Tracy (J & N): “I perceived it as a stressful situation to be in – I was sitting there and the customer expected me to already know the answer.”

Interviews with management strengthened the impression that support work was associated with lower status. Nick (M), MapIT manager, suggested that “support work was not considered part of the career path of highly educated people.”

C. Rotation fairness

We expect also the lower status of CS work to be important in explaining the problem with implementing rotation at MapIT – the rotation was not implemented in a fair manner among the developers. However, mixing firm schedules with project-based work was also problematic. Said Stuart (J): “A project deadline forced me to withdraw from my shift at CS.” Chris (N) commented: “Occasionally we’ve had so much work on the projects that the person believed to be least loaded was put on the CS shift.” Additionally, there were opinions that responsibility of CS should go beyond developers. Noted Andrew (J): “CS is just as much the consultants’ responsibility.” Many participants expressed that they would be more positive towards rotation if the schedule was implemented fairly.

Interviews with management confirmed the difficulty of fairness. All of them raised the issue of implementing a fair rotation schedule in a project-based organization.

D. Learning from rotation

CS work was also associated with useful learning. Helen (N) said: “Working with CS gives insight into the real problematic issues for the customer.” Prior to the trial some developers complained that products were developed with frequent shortcuts that made maintenance difficult. Said Chris (N): “After a few more rounds with rotation I’m sure that the appreciation of developing more maintainable

products will increase.” But Chris (N) added: “I’m not sure, but it would have been easier to improve the understanding of the importance of building maintainable products if there were permanent people in CS. Developers are developers... they don’t express concerns for maintainability.” Being asked specifically about this, he did not believe that the problem of poor maintainability was a result of ASD but simply a consequence of MapIT not recognizing or awarding maintainability.

E. Cost of building general knowledge

Rotation, with these short cycles, received mixed appraisal. First, people warned about the overhead in just getting up to speed. Said Stuart (J): “The short term memory is easily lost. When you re-enter CS you have to learn everything anew.” Some of MapIT’s projects are rather small, and the absence of one developer for the support cycle could be damaging to the progress of the project. David (J) commented: “Being away for two weeks causes the person to lose track of the progress of the project and it may be difficult to regain the comprehension of the software when once returned.” Tracy (J & N) complained: “It takes an awfully long time to sit there to get the overview over things. During the [shift] week you get the job done – alright – but it takes a long time. And the next time you’re on shift... let’s say half a year... you don’t remember how you solved the same enquiry the last time... and you have to start all over. It’s this thing about not mastering what you do, it takes a lot of resources.” Subsequently the efforts needed to get back to speed in the project. Andrew (J) said: “It is similar to having been on vacation – you need some time to get into things again.”

Some developers identified strongly with MapIT’s economical situation and claimed that job rotation to CS was damaging to MapIT. Said Tracy (J & N): “Economically it is directly unprofitable. We could be much more effective on the tasks we need to do, the tasks that are already behind schedule. Instead of sitting there [in the CS room] wasting time.”

F. Value of general knowledge

An important goal with the trial was to evaluate how people reacted to learning about the breadth of products in MapIT’s product portfolio. All participants were asked about whether they perceived specialist knowledge or general knowledge to be most recognized by MapIT. None saw any difference in the recognition of these two knowledge types.

Arguably, the organization is struggling with too many diverging product offerings and two technological platforms. But also the more fundamental aspect of learning about something not directly relevant for the projects in which they participated was received differently.

Prior to the trial MapIT had not implemented any initiatives to develop general knowledge of the product portfolio. Many said that the trial was useful in achieving this. Eric (N), for example, noted: “Working there was very informative with respect to our range of products.” Andrew (J) said: “I don’t want to specialize too much ... I see a vast

range of opportunities in my future career.” But the same developers felt that this time spent on learning was a waste of resources. Teresa (J) said: “The right person would solve this problem much quicker and I didn’t feel useful.” Some found general knowledge to be irrelevant. David (J) claimed: “People from the two camps [Java and .NET] don’t want or need knowledge from the other camp to solve their daily tasks.” The need for specialist knowledge in order to solve CS enquiries were also noted. Said Chris (N): “With the current state of things it is still necessary with the in-depth knowledge on specific topics to feel efficient in there [the CS room]. And it’s not enough to write a few lines of documentation. You are largely dependent on having been a part of the project from the beginning to really understand the system and how to resolve problems.”

A somewhat surprising contradiction was brought forward by Teresa (J): “In the R&D department there is no need for specialists in any area. But I’ve observed specialists associated with one-man projects – then there is only this person who knows the project. If this person quits there is no-one to replace him or her. [...] But it isn’t so that a person has a specific competence which MapIT depends upon. I don’t think that. So if a person is away for a week or so, I think it is fully realistic that someone else can step in.” But specialization, by definition, is identified by people having problems replacing each other on task work. Thus, there appears to be a lack of reflection upon the level of specialist knowledge already manifested by the wide product portfolio.

The increasing awareness of the breadth of the portfolio and the two technological platforms, attributed to the rotation in support work, resulted in reflection upon the challenge of sharing knowledge about the products, the Java and .NET technologies. A very interesting thing happened during the first week of Andrew’s (J) shift in support. He built a Wiki-based support tool to capture frequently used techniques to resolve CS enquiries. All participants contributed and benefitted from the support-wiki. Additionally, Carl (S) claimed that the Wiki “created a common platform for dealing with product issues.”

However, the interviews did not reveal increased insight into potentials for improved technical alignment between products from job rotation.

Interviews with management confirmed the challenge with a large and diverse product portfolio. Nick (M), general manager, said: “We have engaged in an increasing number of projects aiming to build integrated solutions. It’s not easy to stay in control – there are many product variants and many different actors involved. [...] However, we hope that Web Service technology will alleviate this challenge somewhat as both Java and .NET based software can constitute equally valuable service components.” Ken (M), consultant manager, commented: “I think this is a huge challenge for MapIT. We need to focus and better align our products otherwise we’ll not be able to compete.” We asked him if he believed that job rotation could improve the general awareness of importance of technological alignment. He confirmed this, but added that the awareness had to reach the general manager and marketing department somehow.

G. Effects on ‘quagmire’

Several said that institutionalization, from the explicit 2nd line CS role and the CS room, resulted in a more professional CS function which also resulted in better shielding. Andrew (J) said: “Now CS work is planned with resources allocated – it’s not something that comes on top of everything else. And, it is very satisfying to see that many CS enquiries are resolved within this room.” Analysis of CS logs gave support for this claim. A total of 132 CS enquiries were resolved by 2nd line CS and 130 enquiries were resolved by 3rd line CS respectively during the trial. Many said that the volume of CS work became more visible and thus received better recognition by management. However, the decision to reduce staffing from 2 to 1 developer per shift was perceived negatively. Karen (M), just recently appointed development manager, commented that the decision was made based on internal reviews of initial logs which showed that there wasn’t enough CS work to justify 2 developers on each shift. Also, the CS manager commented that he did not perceive CS work as income-generating (income-generating work was attributed developers).

H. Bureaucracy

As mentioned in subsection IV.A, all developers wanted more formalism through a permanent 2nd line CS staff. They believed this would give stronger ownership and commitment to the function. This, it was claimed, would result in benefits such as more strongly expressed quality requirements, further improved shielding and more efficient CS.

Interviews with management confirmed the willingness to increase bureaucracy with respect to CS. Although it was expressed using more neutral language. Nick (M), general manager, explained: “It is important to ‘hit the mark’ with CS staffing so that we don’t waste resources in there [in CS]. It would probably be more efficient from a competence perspective to rather have one or two permanent staff on CS that would get up to speed quicker.”

V. DISCUSSION

Action research has its epistemological basis in a preference to knowing how over knowing that. This is reflected in AR’s central position “the best understanding of a phenomenon is obtained when trying to change it.” [13] But organizational change takes time. The trial reported here lasted only 18 weeks so we cannot expect that the full potentials or all disadvantages of job rotation are revealed. The findings suggest that introducing job rotation may bring benefits [26] but it should be done very carefully. It is worth noting that the shielding effect was significant. Although the measurements and duration of logging didn’t allow for deep analysis, the logs showed that more CS enquiries were resolved by the 2nd line CS person on shift than those resolved by developers outside the shift for the duration of the trial. (132 versus 130.) This implies that the number of ‘quagmire’ requests to the remaining developers was reduced by roughly 50 % by the rotation.

We now address each of the three research questions raised in Section II.

A. How can job rotation contribute to the building of general knowledge in an organization with a prevalence of specialist knowledge?

Job rotation has been held to represent learning organizations and contribute to innovative performance [7, 17]. In this case, the Wiki resulted from the need to quickly retrieve and accumulate ‘problem-solving tricks’ for the whole product portfolio. The Wiki helped share useful knowledge for all [12]. Furthermore, the need to improve maintainability of the products has become more visible and the importance of CS has improved – thus improving appreciation for other organizational concerns [7].

But the findings from this research shows that only a minority of the participants wanted to continue with the rotation schedule. In fact, all participants were in favor of a more bureaucratic CS organization. Thus, job rotation should probably be implemented differently in order to accomplish the sustained building of general knowledge. One suggestion could be to use a scheme that is based on voluntary participation. Implementing job rotation with short cycles in a fair manner in a project-based organization is difficult.

B. What impact can the introduction of job rotation have on enabling double-loop learning in a software organization?

Enabling double-loop learning is hard, involves change to fundamental thinking and may take considerable time. For these, and other reasons, efforts often fail [2]. It would be unreasonably harsh to say that this trial of job rotation failed to enable double-loop learning. In fact, as we noted in Subsection V.A above, initial phases of double-loop learning can be traced to the trial. The increased appreciation of building more maintainable products and the support-wiki can be interpreted as a fundamental change in norms and strategies used to deal with occurring problems [2].

However, with the termination of job rotation this source of feedback disappears and double-loop learning must be accomplished by other means. The failure to sustain double-loop learning in this case can be partly attributed the difficulty of appreciating the effect of changes that initially seem to only incur extra costs for the individual and the organization.

Besides, CS work was considered more stressful and this can contribute to less time for reflection upon issues such as exploration of new ideas [20] and improved technology alignment between products. The decision made by MapIT only 4 weeks into the trial to reduce staffing from 2 to 1 developer on shift implied that Java programmers would have to resolve enquiries relating to products developed in .NET, and vice versa. This naturally decreased the potential of the support room to function as the arena for rich learning opportunities we first envisaged. It also increased the feeling of stress for those on the shift. Besides, participants objected strongly to this, mostly because Java applications represented a low fraction of the CS enquiries but also because the

technological gap was considered too large. It should also be considered whether it is actually desirable to aim for this level of general knowledge across such a dispersed product portfolio. Additionally, it was both done and perceived as a cost-saving decision and likely contributed to the view of CS as merely a cost for MapIT rather than a source for useful knowledge of customer concerns [32].

C. Can the introduction of job rotation illuminate other challenges related to organizational learning in software organizations?

ASD establish its efficiency ideal in flexibility, responsiveness, lightweight documentation and effective teams communicating informally [9, 20]. Existing theory shows that this efficiency ideal is better maintained with a significant degree of general knowledge. The empirical results from this investigation showed that all participants wanted a more bureaucratic organization of CS than one based on job rotation. In fact, the desire to be efficient at given roles was strongly expressed among the participants in this trial and contributed to the abandonment of job rotation as a vehicle for building general knowledge. Despite the fact that general knowledge was found interesting and valuable among most participants it was not enough to motivate continued rotation. This supports existing management and learning theory, namely that specialization supports peoples’ need for task proficiency [12] and efficiency [15]. At the organizational level, efficiency through bureaucracy was implied as a goal by management’s use of the machine metaphor to describe an ideal state. Furthermore, in this case, all of the interviewed people wanted more bureaucratic organization of the CS function to further improve efficiency, predictability and ability to deliver ‘professional’ CS. The findings suggest that using job rotation to CS for the building of general knowledge in order to ease adoption of ASD can be very challenging. Even more so if the software organization has a wide product portfolio like MapIT.

Furthermore, it proved difficult to implement even these relatively short shifts in the project-based organization [30]. Although the length of the shift had been adjusted to the duration of the Scrum iteration, the lack of long-term plans in Scrum made the rotation more difficult – it was close to impossible to predict when important stages in the projects would occur and the schedule had to be changed. If CS work is considered less interesting, implementing the schedule in a fair manner becomes more important.

Thus, even for organizations relatively proficient with ASD the efficiency ideals of former bureaucratic work organization may be prevalent.

VI. CONCLUSIONS

Generalization from single-case investigations like this one should be done with the utmost care. What seems reasonable to claim based from this investigation is that it does take time to build a learning organization culture and gain benefits from general knowledge. Attempts to build general knowledge through job rotation should be done with consideration to peoples’ inherent desire to complete their

tasks efficiently. Job rotation to CS does bring increased appreciation for CS issues such as maintainability and certain customer usage scenarios. It appears as if the efficiency ideals of former bureaucratic practices for organization of work are not easily altered. Further research is necessary to strengthen the empirical evidence. Continued research focusing on different adoption models of job rotation would also be useful in order to explore correlations between voluntary participation schemes and their sustainability in the agile software organization.

Although some benefits such as clearer understanding of customers' requirements, maintainability as an important product quality and inventions in tools etc. may appear quickly, deeper appreciations of opportunities for technological alignment and future products may take longer time. Thus, management must be willing to accept the cost of building general knowledge. Individual differences may justify schemes based on voluntary participation, but this should be weighted against the potential challenge of having to adjust the status of rotation work.

ACKNOWLEDGMENT

The work in this paper was supported by the Research Council of Norway through the project EVidence based Improvement of SOFTWARE engineering (EVISOFT). I am grateful to the involved people at MapIT and fellow researcher Torgeir Dingsøyr at SINTEF ICT for kind cooperation.

REFERENCES

- [1] J.M. Allwood and W.L. Lee, "The impact of job rotation on problem solving skills", *International Journal of Production Research*, 2004. **42**(5): pp. 865-881.
- [2] C. Argyris and D.A. Schön, *Organizational learning II: Theory, method, and practice*. 1996, Reading, Massachusetts: Addison-Wesley. 305.
- [3] R. Baskerville and T.A. Wood-Harper, "Diversity in information systems action research", *European Journal of Information Systems*, 1998.
- [4] B. Boehm and R. Turner, "Management challenges to implementing agile processes in traditional development organizations", *IEEE Software*, 2005. **22**(5): pp. 30-39.
- [5] P.R. Carlile, "Transferring, translating, and transforming: An integrative framework for managing knowledge across boundaries", *Organization Science*, 2004. **15**(5): pp. 555-568.
- [6] S.R. Clegg, D. Courpasson and N. Phillips, *Power and organizations*. Foundations for organizational science. 2006, London: SAGE Publications. 457.
- [7] M.M. Cosgel and T.J. Miceli, "Job rotation: Cost, benefits and stylized facts", *Journal of Institutional and Theoretical Economics*, 1999. **155**(2): pp. 301-320.
- [8] R.M. Davison, M.G. Martinsons and N. Kock, "Principles of canonical action research", *Information Systems Journal*, 2004. **14**(1): pp. 65-86.
- [9] T. Dybå and T. Dingsøyr, "Empirical studies of agile software development: a systematic review", *Information and Software Technology*, 2008. **50**(9-10): pp. 833-859.
- [10] A.C. Edmondson and S.E. McManus, "Methodological fit in management field research", *The Academy of Management Review*, 2007. **32**(4): pp. 1155-1179.
- [11] D. Gebert, S. Boerner and E. Kearney, "Cross-functionality and innovation in new product development teams: A dilemmatic structure and its consequences for the management of diversity", *European Journal of Work and Organizational Psychology*, 2006. **15**(4): pp. 431 - 458.
- [12] R.M. Grant, "Prospering in dynamically competitive environments: organizational capability as knowledge integration", *Organization Science*, 1996. **7**(4): pp. 375-387.
- [13] D.J. Greenwood and M. Levin, *Introduction to action research: Social research for social change*. 2nd Edition ed. 2007: SAGE Publications. 274 pp.
- [14] O. Hauptman and K.K. Hirji, "Managing integration and coordination in cross-functional teams: an international study of Concurrent Engineering product development", *R & D Management*, 1999. **29**(2): pp. 179-191.
- [15] D.A. Kolb, *Experiential learning: Experience as the source of learning and development*. 1984, New Jersey: Prentice-Hall.
- [16] C. Larman and V. Basili, "Iterative and incremental development: A brief history", *IEEE Computer*, 2003. **36**(6): pp. 47-56.
- [17] B.-Å. Lundvall and P. Nielsen, "Knowledge management and innovative performance", *International Journal of Manpower*, 2007. **28**(3-4): pp. 207-223.
- [18] G. Melnik and F. Maurer. *Direct verbal communication as a catalyst of agile knowledge sharing*. in *Agile Development Conference (ADC'04) 2004*. 2004. Salt Lake City, Utah.
- [19] N.B. Moe, T. Dingsøyr and T. Dybå. *Understanding self-organizing teams in agile software development*. in *In proceedings of 19th Australian Conference on Software Engineering*. 2008: IEEE Computer Society.
- [20] S. Nerur and V. Balijepally, "Theoretical reflections on agile development methodologies: The traditional goal of optimization and control is making way for learning and innovation", *Communications of the ACM*, 2007. **50**(3): pp. 79-83.
- [21] S. Nerur, R. Mahapatra and G. Mangalaraj, "Challenges of migrating to agile methodologies", *Communications of the ACM*, 2005. **48**(5): pp. 72-78.
- [22] I. Nonaka and H. Takeuchi, *The knowledge-creating company: How Japanese companies create the dynamics of innovation*. 1995, New York: Oxford University Press. 284.
- [23] J. Ortega, "Job rotation as a learning mechanism", *Management Science*, 2001. **47**(10): pp. 1361-1370.
- [24] P.N. Robillard, "The role of knowledge in software development", *Communications of the ACM*, 1999. **42**(1): pp. 87-92.
- [25] D.L. Rulke and J. Galaskiewicz, "Distribution of knowledge, group network structure, and group performance", *Management Science*, 2000. **46**(5): pp. 612-625.
- [26] I. Rus and M. Lindvall, "Knowledge management in software engineering", *IEEE Software*, 2002. **19**(3): pp. 26-38.
- [27] E. Salas, D.E. Sims and C.S. Burke, "Is there a 'big five' in teamwork?" *Small Group Research*, 2005. **36**(5): pp. 555-599.
- [28] M. Song, H. van der Bij and M. Weggeman, "Factors for improving the level of knowledge generation in new product development", *R & D Management*, 2006. **36**(2): pp. 173-187.
- [29] G.I. Susman and R.D. Evered, "An assessment of the scientific merits of action research", *Administrative Science Quarterly*, 1978. **23**: pp. 582-603.
- [30] K.E. Söderquist, "Organizing knowledge management and dissemination in new product development: Lessons from 12 global corporations", *Long Range Planning*, 2006. **39**(5): pp. 497-523.
- [31] C.A. Un, "Managing the innovators for exploration and exploitation", *Journal of Technology Management and Innovation*, 2007. **2**(3): pp. 4-20.
- [32] X. Wang, *Organizing to be adaptive: a complex adaptive systems based framework for software engineering processes*, PhD thesis, School of management, University of Bath, 2007.
- [33] M. Weber, *Makt og byråkrati*. 2000, Trondheim: Gyldendal.
- [34] H.D. Yang, H.R. Kang and R.M. Mason, "An exploratory study on meta skills in software development teams: antecedent cooperation skills and personality for shared mental models", *European Journal of Information Systems*, 2008. **17**(1): pp. 47-61.